



Instituto Tecnológico de Aeronáutica - ITA
CM-204 - Aprendizado Profundo
Aluno: Ulisses Lopes da Silva

Laboratório 06 - Processamento de Linguagem Natural com RNN

1 Breve explicação em alto nível do que foi implementado

1.1 Construção do modelo recorrente

A classe `RecurrentModel` foi construída com três camadas. A primeira é uma camada de *embedding*, que converte cada palavra (representada por um número inteiro) em um vetor de 32 valores. A segunda é uma camada recorrente que pode ser de três tipos diferentes — RNN básica, LSTM ou GRU — escolhida pelo parâmetro `cell_type`; em todos os casos, ela possui 32 unidades. A terceira é uma camada totalmente conectada com apenas um neurônio, responsável por produzir a saída final.

```
self.embedding = nn.Embedding(max_features, embed_dim, padding_idx=0)

if cell_type == "lstm":
    self.rnn = nn.LSTM(embed_dim, num_units, batch_first=True)
elif cell_type == "gru":
    self.rnn = nn.GRU(embed_dim, num_units, batch_first=True)
else:
    self.rnn = nn.RNN(embed_dim, num_units, batch_first=True)

self.fc = nn.Linear(num_units, 1)
```

Listing 1: Construtor do modelo recorrente

1.2 Passagem direta (*forward*)

No método `forward`, o texto de entrada (uma sequência de números inteiros representando palavras) é primeiro convertido em vetores pela camada de *embedding*. Esses vetores passam pela camada recorrente, que processa a sequência palavra por palavra. Apenas a saída referente à última palavra é utilizada, pois ela resume todo o contexto da frase. Por fim, essa saída passa pela camada final e pela função sigmoide, produzindo um valor entre 0 e 1 que representa a probabilidade de a avaliação ser positiva.

```
x = self.embedding(x)
output, _ = self.rnn(x)
```

```
output = output[:, -1, :]  
return torch.sigmoid(self.fc(output))
```

Listing 2: Forward do modelo recorrente

1.3 Avaliação do modelo

A função `evaluate_model` mede o desempenho da rede no conjunto de teste. Os dados são convertidos para tensores e organizados em lotes. Em seguida, o modelo faz previsões para cada lote e a função calcula a perda média e a acurácia (porcentagem de previsões corretas) considerando todas as amostras.

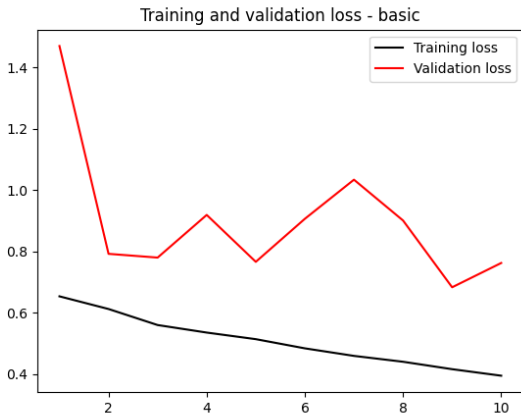
```
x_test_t = torch.tensor(x_test, dtype=torch.long)  
y_test_t = torch.tensor(y_test, dtype=torch.float32).unsqueeze(1)  
test_loader = DataLoader(TensorDataset(x_test_t, y_test_t), batch_size=  
    batch_size)  
  
test_loss = 0.0  
test_correct = 0  
test_total = 0  
  
with torch.no_grad():  
    for xb, yb in test_loader:  
        xb, yb = xb.to(device), yb.to(device)  
        preds = model(xb)  
        test_loss += criterion(preds, yb).item() * len(xb)  
        test_correct += ((preds > 0.5).float() == yb).sum().item()  
        test_total += len(xb)  
  
test_loss = test_loss / test_total  
test_acc = test_correct / test_total
```

Listing 3: Avaliação do modelo

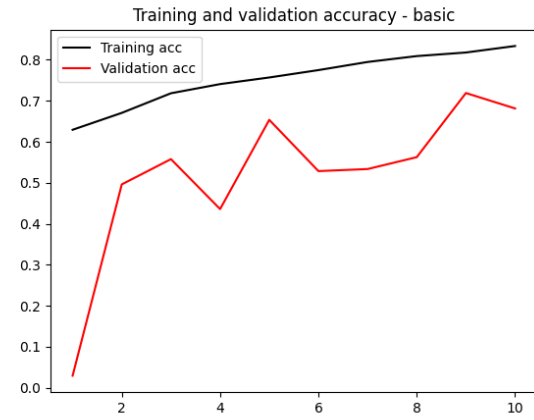
2 Figuras Comprovando Funcionamento do Código

A seguir são apresentadas as curvas de perda e de acurácia ao longo das épocas de treinamento para cada um dos três tipos de célula recorrente utilizados.

2.1 RNN Básica



(a) Perda - RNN básica.



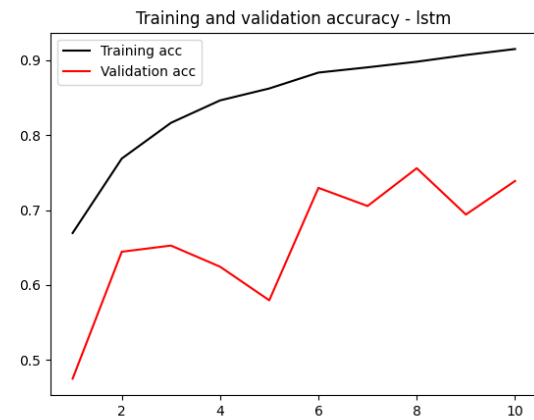
(b) Acurácia - RNN básica.

Fig. 1: Curvas de treinamento e validação para a RNN básica.

2.2 LSTM



(a) Perda - LSTM.



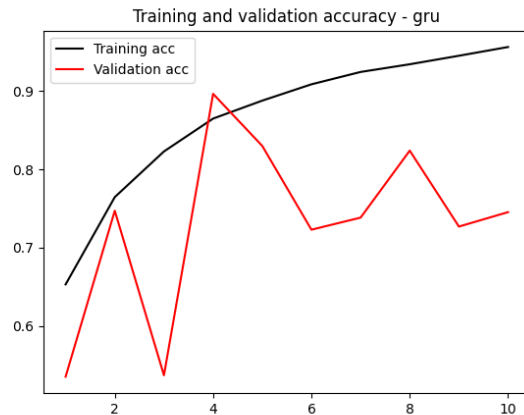
(b) Acurácia - LSTM.

Fig. 2: Curvas de treinamento e validação para a LSTM.

2.3 GRU



(a) Perda - GRU.



(b) Acurácia - GRU.

Fig. 3: Curvas de treinamento e validação para a GRU.

2.4 Resultados no conjunto de teste

A Tabela 1 resume o desempenho dos três modelos no conjunto de teste, utilizando os melhores pesos salvos durante o treinamento.

Tabela 1: Perda e acurácia dos modelos no conjunto de teste.

Modelo	Perda	Acurácia
RNN básica	0,5244	0,7806
LSTM	0,4066	0,8277
GRU	0,3967	0,8242

3 Discussão dos Resultados

- **Curvas de treinamento:** As Figs. 1, 2 e 3 mostram que todos os três modelos aprendem ao longo das épocas: a perda de treino cai de forma suave e a acurácia de treino sobe consistentemente. Por outro lado, as curvas de validação apresentam formas bem ruidosas em todos os casos, o que é esperado por dois motivos: (i) o conjunto de validação é menor que o de treino (divisão do conjunto inicial de treino na proporção 80/20 para *test/val*), tornando as métricas mais ruidosas; e (ii) os modelos começam a sofrer *overfitting* após algumas épocas, especialmente a LSTM e a GRU, em que a perda de validação chega a subir enquanto a de treino continua caindo.
- **Comparação entre as células:** A RNN básica apresenta o pior desempenho de treino entre as três: sua perda decai mais lentamente e a acurácia de treino atinge um patamar

mais baixo. A LSTM e a GRU se mostram bem mais eficientes, atingindo acurácias de treino acima de 80%. Esse comportamento é consistente com a teoria: a RNN básica sofre com o problema do *vanishing gradient* em sequências longas (no caso, frases de até 500 palavras), o que dificulta o aprendizado de dependências entre palavras distantes. Já a LSTM e a GRU foram projetadas justamente para contornar esse problema, por meio de mecanismos de portas (*gates*) que controlam o fluxo de informação ao longo da sequência.

- **Resultados no teste:** A Tabela 1 confirma o que se observa nas curvas: a LSTM e a GRU têm desempenho superior à RNN básica, com acurácia de teste em torno de 82%, enquanto a RNN básica fica em 78%. Entre LSTM e GRU, os resultados são praticamente equivalentes — a GRU teve perda ligeiramente menor (0,3967 contra 0,4066), enquanto a LSTM teve acurácia um pouco maior (0,8277 contra 0,8242). Essa equivalência também condiz com a teoria, pois a GRU é uma versão simplificada da LSTM, com menos parâmetros e funcionamento semelhante, e na prática costuma alcançar resultados muito próximos. Como ambos os modelos ultrapassaram o limiar de 80% sugerido na documentação do código, considera-se que o experimento foi bem sucedido.